Greedy Piggies Contribution Presentation

Created by
Callum Wade
2404781

Tools

- Jenkins
- Jenkins Webhook
- GitHub Webhook
- ClickUp Webhook

What is Jenkins

- An open source automation tool that automates repetitive tasks such as building the project
- It builds the game at 10pm every day
- Whenever the pipeline is run it:
  - Builds the game
  - Uploads the build to GitHub
  - Uses the Jenkins webhook to send a message to Discord that contains a link to the build, the status of the build and the build number
  - Updates the data tables that contain the build data

How it works

- Jenkins runs on my home PC and runs a groovy script on the Greedy Piggies pipeline at 10pm everyday
- This pipeline then accesses the staging branch from GitHub and builds the project through the Unreal Engine build tool.
- After the code is run, it will either say success or failed
- Success means it has done everything it is supposed to do, as stated in the previous slide
- Failed means that there is either a problem with the pipeline or that something is wrong with the project, which by that point I would resolve the problem by looking through the Jenkins output log

How does this help the development

- Jenkins being integrated into the project means that no one is required to manually build the game periodically.
- Instead Jenkins does this at a specific time every day without the need for human input, removing the chance of any human error.
- This also allows errors to be found early, allowing them to be focused on and fixed before development continues, as at the end of the day, the game building is the most important thing.

Jenkins Webhook

- Allows Jenkins to send a message to the Discord server to notify channel members the status of the staging branch as well as providing access to the recent build
- This allows the developers to keep track of staging and allows easy access to the most recent builds of the game.

GitHub Webhook

- Whenever a change is completed anywhere in the repository, GitHub sends a message to the Discord server
- This message includes:
  - The name of the change
  - Description of the change
  - Who made the change
  - What was changed
  - A link to view the change on GitHub.com

GitHub

How does this help the development

- Allows everyone to easily get notified whenever anyone makes a change in the repository
- Provides easier access to push reviewers as they can just click on a change on the server and will be sent straight to the change on GitHub's website with a more in-depth view of what has been changed

ClickUp Webhook

- Whenever a task is created or the status of a task is changed on ClickUp, a message is sent on the set Discord channel.
- These messages include:
  - Who created/changed the task
  - Who the task is assigned to
  - The name of the task
  - The current and previous status of the task
  - Which workspace and section within the workspace the task is in
  - A link to the workspace and to the task
- This webhook was created for the developers and designers, each with their own separate Discord channel
- The main purpose of this tool was to motivate people to use the ClickUp and to also keep people notified on what has been completed or what needs to be completed

How does this help the development

- Keeps people notified of what tasks are still to be completed and of which tasks have been completed
- Reminds people and motivates people to keep using the ClickUp and keeping it up to date

1/3 DONE
MEME BREAK

READING
GAME DEV
MEMES

MAKING
GAME DEV
MEMES

AND ALSO GAMES
AND MARKETING THEM
SEND HELP PLEASE

My roles within this project

- GitHub Push Reviewer (Responsible)
- Automated Builds (Responsible)
- Design Liaison (Responsible)
- Ticket Reviewing (Accountable)
- Clickup (Accountable)
- Multiplayer (Accountable)

GitHub Push Reviewer

How I fulfilled this role:

- Implemented a GitHub webhook to the Discord server to easy access and viewing of new changes and pushes to the repository
- Kept track of changes
- Used Jenkins to fix any pushes that caused errors to any of the primary branches

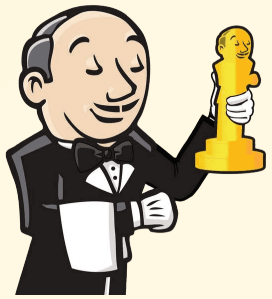
Automated Builds

How I fulfilled this role:

- Implemented Jenkins into the project to automatically build the game every day
- Added a webhook for Jenkins so that everyone would know when the game built and the status of the build
- Further worked on Jenkins to upload the build that is created to GitHub with a new tag so that new and old versions of the build are accessible online
- Added to the pipelines code so that Jenkins uses Python to create and update data tables that display the time to build and status of each build

What I could have done better:

- Implementing Jenkins sooner into the project
- Fixing the issue that cause it to not build automatically some days



Design Liaison

How I fulfilled this role:

- Provided all the designers with access to the repository and set their permission levels
- Communicated with the designers whenever something needed clearing up

What I could have done better:

- Checking in on the designers more often

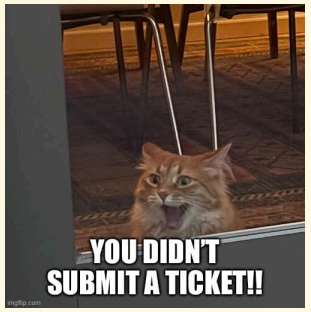
Why I felt this role didn't work:

- Due to the high amount of departments and people working on the project, everyone would just speak to the specific designers themselves if they needed to clear something up as it is much easier then asking a liaison, which makes sense and is why I felt that this role didn't work for this project

Ticket Reviewing

How I fulfilled this role:

- Contacted and helped anyone who I could that uploaded a ticket that hadn't already been accepted



Clickup

How I fulfilled this role:

- Created a ClickUp webhook that would notify the departments that used it when a change was made in their own ClickUp workspace

ClickUp

Multiplayer

How I fulfilled this role:

- Worked on the original multiplayer and created the server browser to allow players to find and connect to other players through a server list or by joining through Steam friends

2/3 DONE
MEME BREAK

Contributions to the development of Greedy Piggies

- The majority of my work was focussed on the UI menu screens and the multiplayer of Greedy Piggies as they were tied very closely together
- I started by creating the first version of the multiplayer script that allowed players to connect by LAN in the server browser and create game screens and then I moved onto helping with getting online connections working whilst also creating more menus and improving the ones that I had already made with more game ready features.

Menu Screens

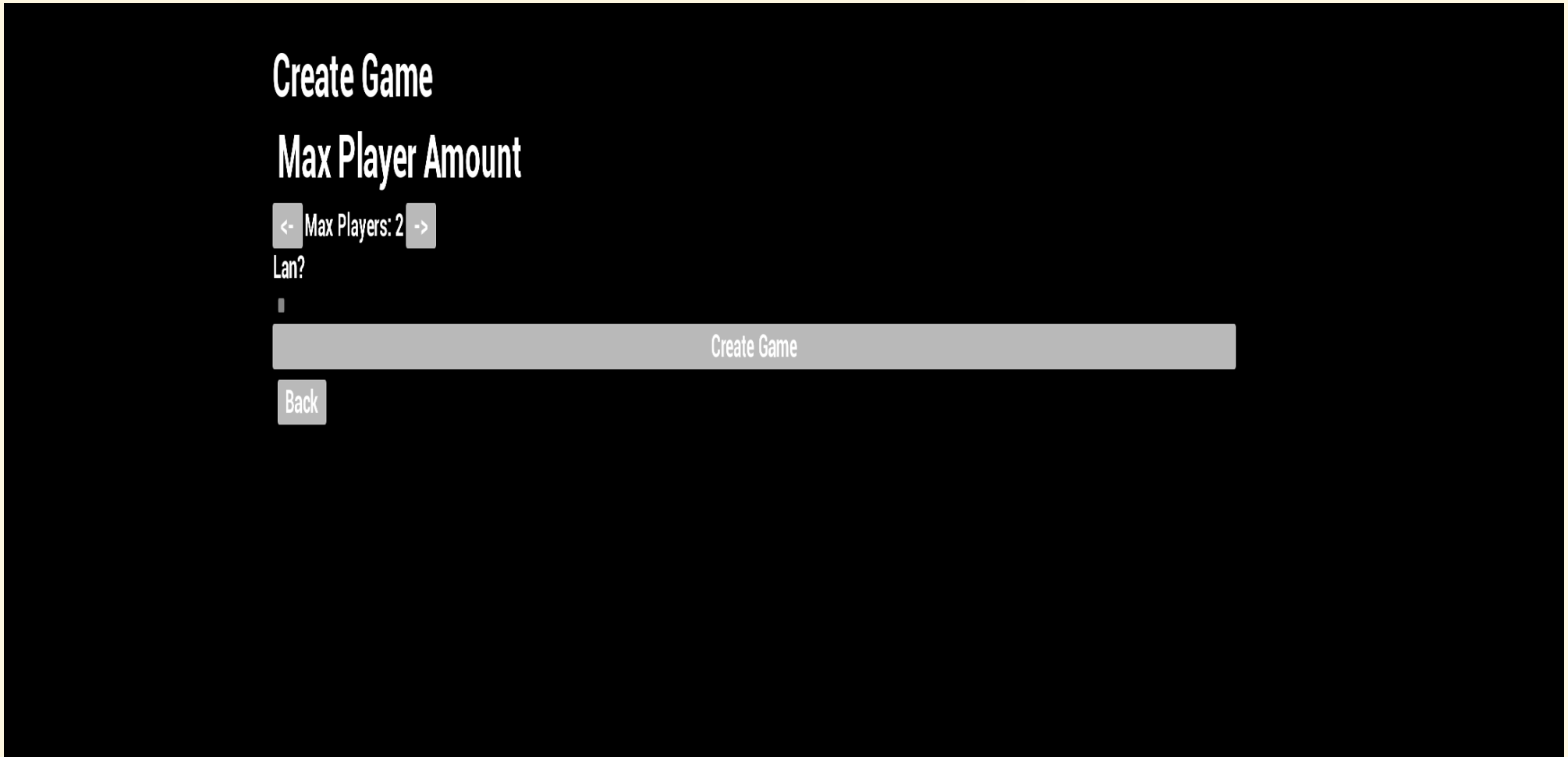
- The menus were originally created to test the multiplayer but ended up being used as the bases that the menus in the game were designed using
- These menus included:
  - Main Menu Prototype
  - Create Game
  - Server Browser
  - Server Browser Item
  - Pause Menu Prototype
  - Character Select Screen

Main Menu Prototype

- The main menu that I created allows the player to choose between creating a lobby or going to the server browser to search for one

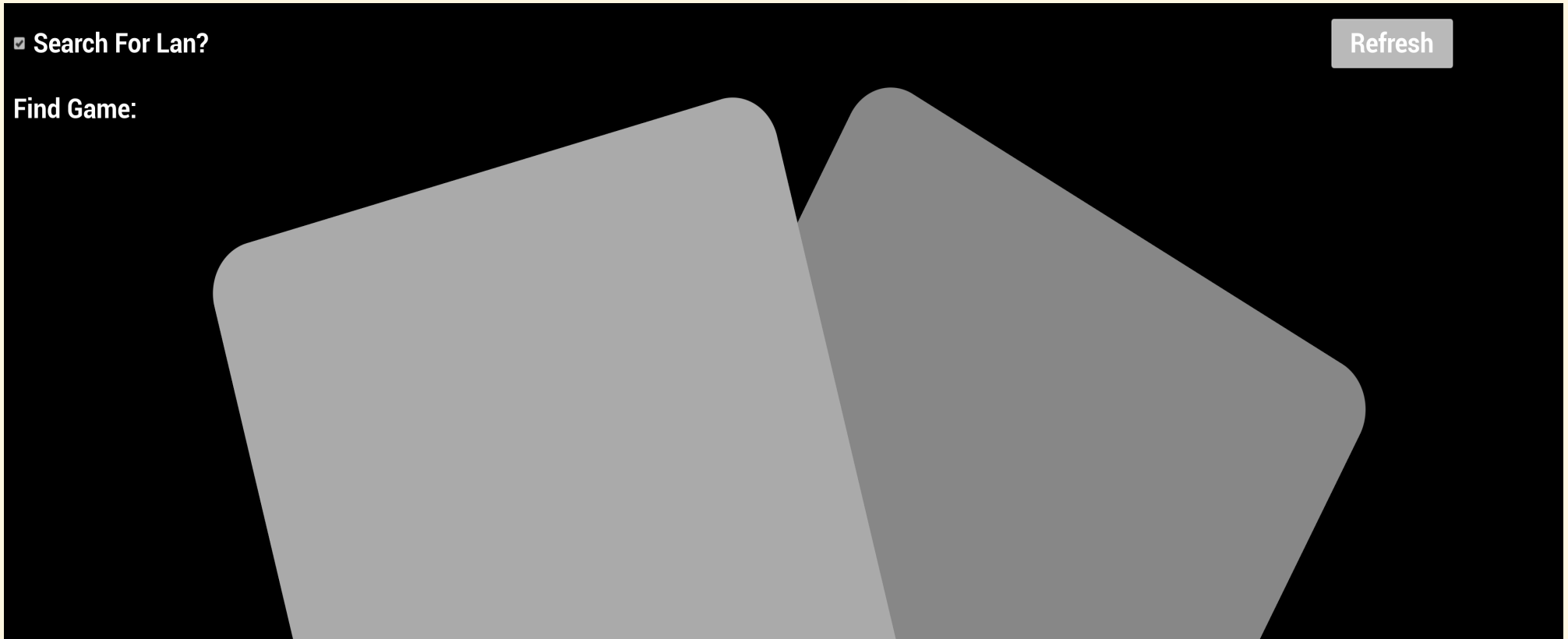
Create Game

- The create game menu lets the player decide the maximum amount of player that can join their lobby as well as if the lobby is online or LAN.



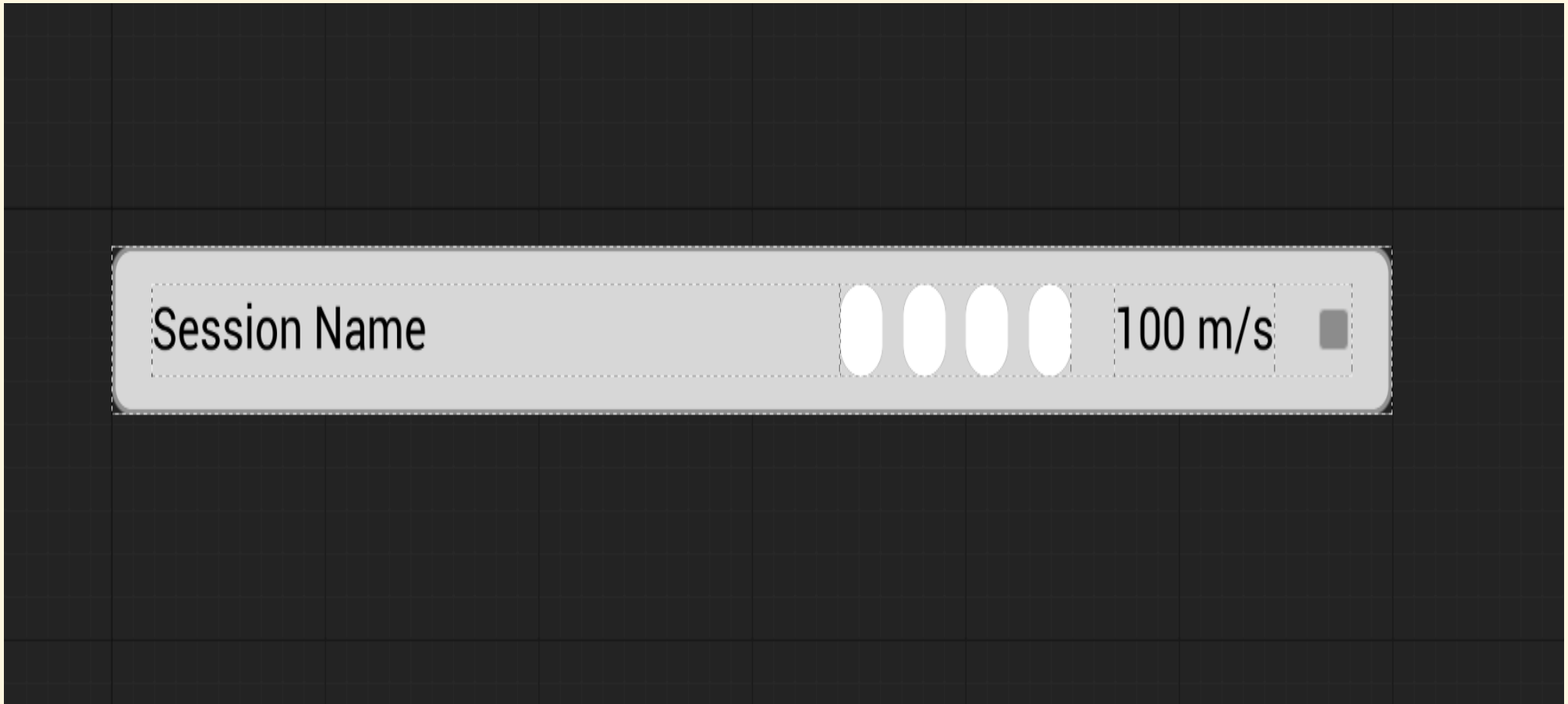
Server Browser

- The server browser searches for lobbies and displays each found lobby as a server browser item
- Buttons at the top of the menu allow players to search for only LAN lobbies and also refresh the lobby list



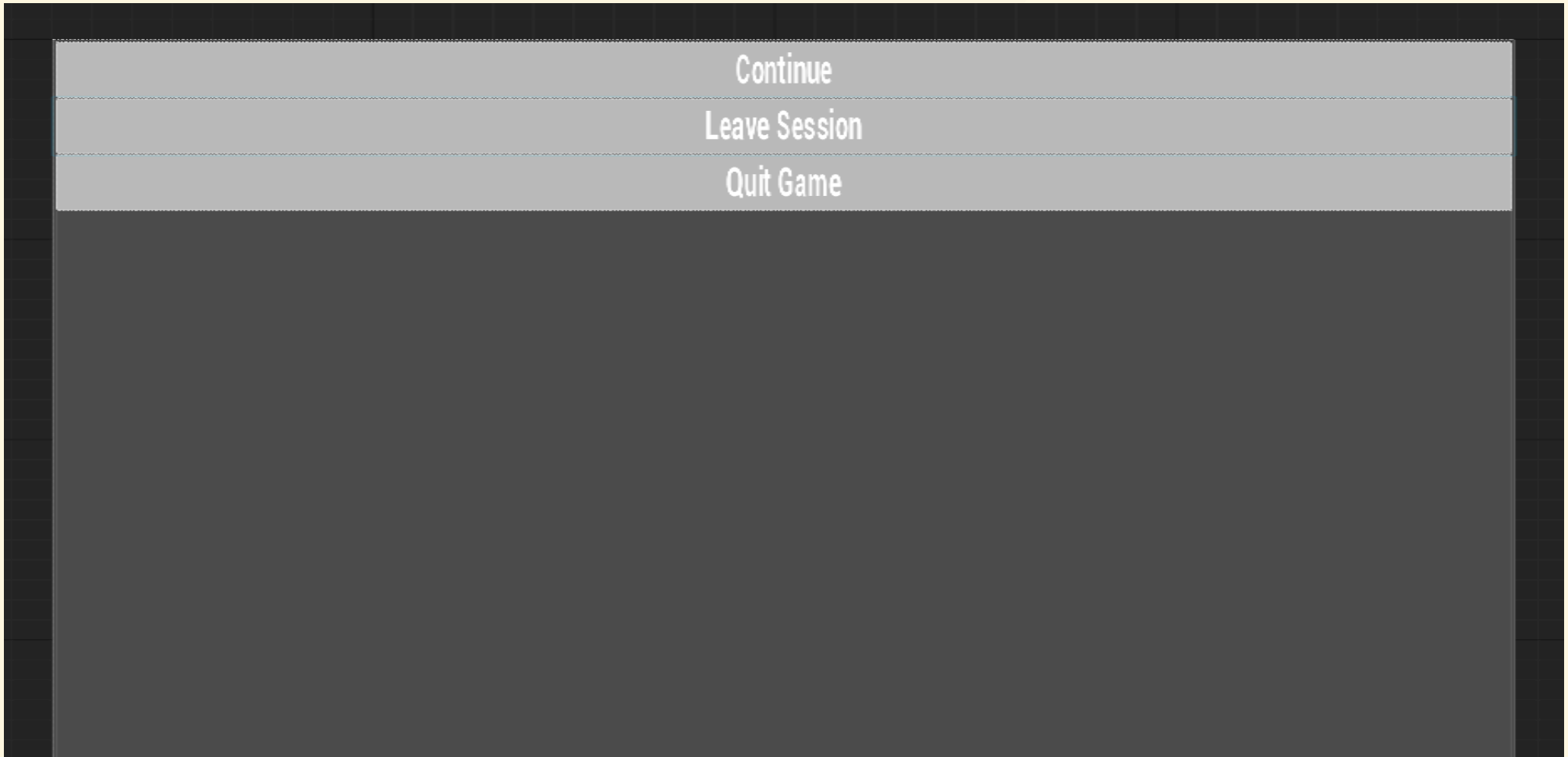
Server Browser Item

- The server browser item appears in the server browser once for each lobby found and displays the lobby name, the amount of players in the lobby, the connection/ping and a check box to choose that lobby



Pause Menu Prototype

- The pause menu has buttons that resume the game, leave the current session and quit the whole game



Character Select Screen

- The characters select screen is used as a pre game lobby to allow players to pick their characters
- Once players have picked their character, they can ready up. Once all players are ready, they are sent into the main game
- This menu was originally created as just a basic pre game lobby as a possible solution to fix a problem with players not seeing each other in the game but ended up turning into a working character select screen